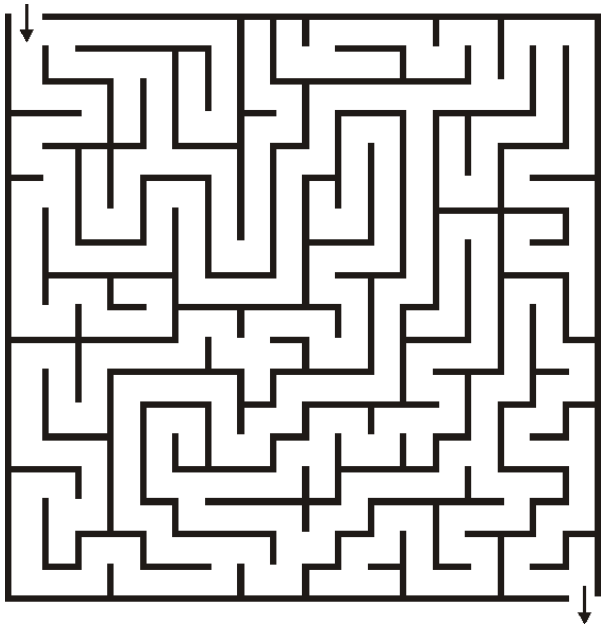
MAZE SOLVING PROJECT

-Anissa Sollman, Mani Botla, Nick Liverett

OVERVIEW

- Finding a path from a starting position to a destination within a maze.
- Represented using a 2D array or matrix.
- Helps develop key search algorithms (DFS, BFS, A*)
- Real-world use: routing, navigation, logistics, GPS systems



RESEARCH

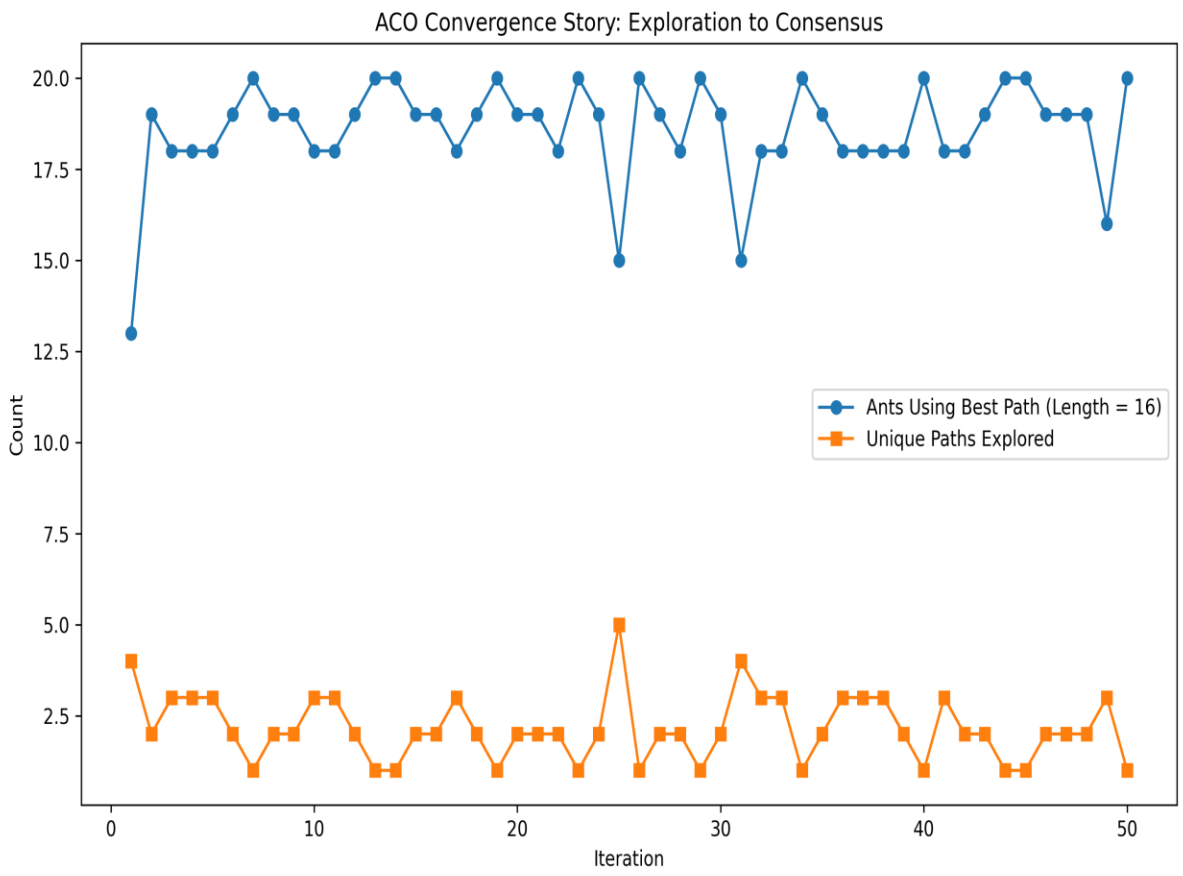
- Goal : Compare maze-solving algorithms to find the most efficient shortest-path method.
- Four algorithms were used:
  - Breadth First Search (BFS)
  - Depth First Search (DFS)
  - A* Search
  - Ant colony optimization(ACO)
- Expectations:

- Expected A* to be fastest and efficient.
- Expected DFS to take longer and be less optimal..

EXPERIMENTATION

- DFS, BFS, and A* tested on all maze types; ACO worked only on small mazes
- ACO influenced by pheromone strength and several other factors.
- ACO much slower due to many ants and iterations.
- Adjusted factors made ACO too deterministic on small mazes i.e showed very fast convergence.

RESULTS



- Ants quickly begin choosing the best path (length 16).
- Unique paths explored drop to very low numbers
- Shows strong convergence
- Stable behavior
- Best- path usage stays high across iterations

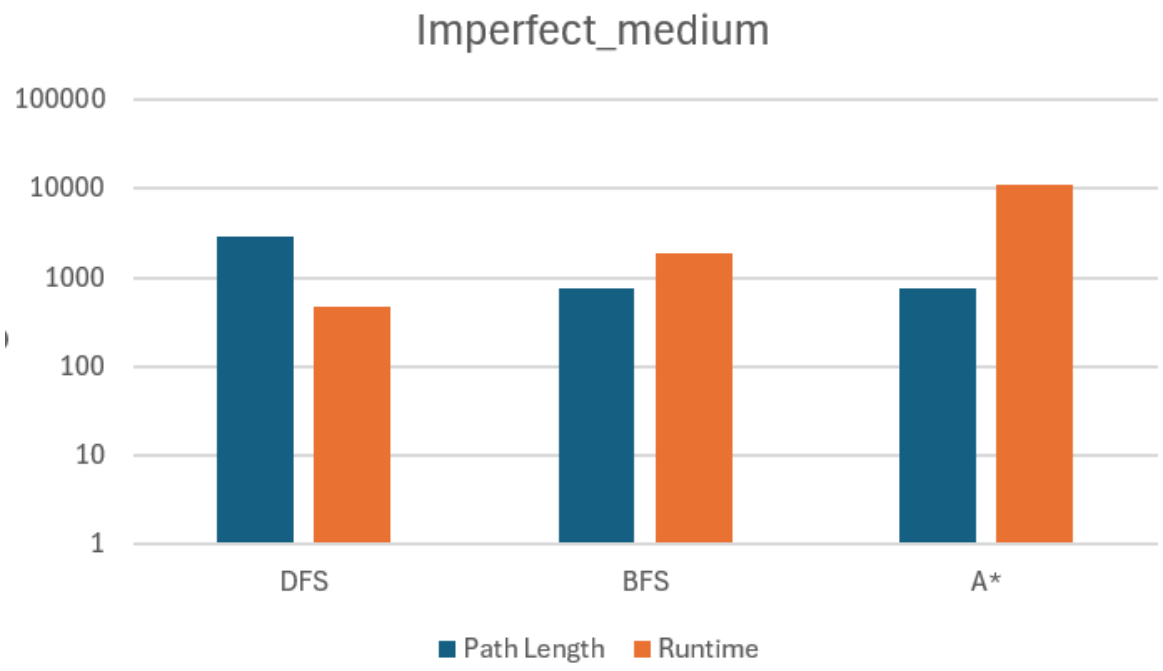
ACO CONVERGENCE GRAPH

To display convergence, we want to see less unique paths and more ants choosing the best path over the iterations

Table 3: ACO convergence Results

Iteration	Best path length	Unique paths	Ants that used best path	Successful Ants	Average path length
1	16	5	13	20/20	41.2
2	16	1	20	20/20	16.0
3	16	3	18	20/20	19.4
4	16	3	17	20/20	21.0
5	16	1	20	20/20	16.0
6	16	1	20	20/20	16.0
7	16	1	20	20/20	16.0
8	16	1	20	20/20	16.0
9	16	2	19	20/20	24.0
10	16	2	18	20/20	26.0
11	16	3	18	20/20	28.0
12	16	3	18	20/20	38.0
13	16	1	20	20/20	16.0
14	16	2	19	20/20	26.0
15	16	1	20	20/20	16.0
16	16	2	19	20/20	26.0
17	16	2	19	20/20	26.0
18	16	1	20	20/20	16.0
19	16	4	17	20/20	34.5
20	16	4	17	20/20	34.5
21	16	1	20	20/20	16.0
22	16	1	20	20/20	16.0
23	16	1	20	20/20	16.0
24	16	2	19	20/20	24.0

GRAPHS


Imperfect_medium
100000
10000
1000
100
10
1
DFS
BFS
A*
Path Length
Runtime

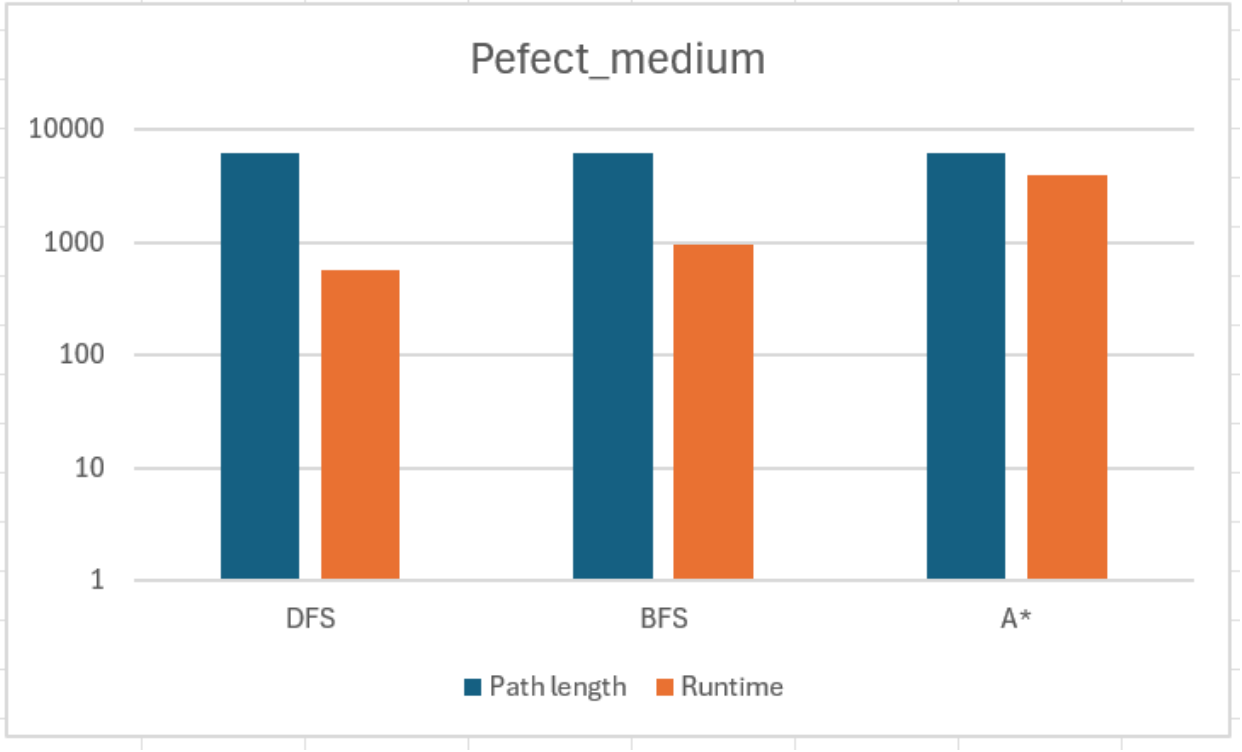

Pefect_medium
10000
1000
100
10
1
DFS
BFS
A*
Path length
Runtime

Table 1: Results from using imperfect_small dataset

Algorithm	Found Path	Path Length	Runtime (in microseconds)	Optimal (Yes/No)
DFS	Yes	16	12	Yes
BFS	Yes	16	12	Yes
A*	Yes	16	26	Yes
Ant colony optimization (ACO)	Yes	16	67227	Yes

Table 2: Maze Size Vs Run Time

Algorithm	imperfect_small	perfect_small	imperfect_medium	perfect_medium
DFS	12	32	472	571
BFS	12	46	1883	955
A*	26	83	11288	3919

ANALYSIS

- The results show that no single algorithm works best for every maze size.
- For instance, in imperfect_small dataset, BFS and DFS are the most practical choices.
- DFS performed best for runtime efficiency.
- BFS performed best for guaranteed shortest path finding.
- A* is useful when a strong heuristic exists.
- ACO was best for optimization analysis.

ANALYSIS

- In ACO convergence story, only 13 ants used the best path in the first iteration.
- This shows that colony initially explored multiple possible routes instead of immediately agreeing on one solution.
- By second iteration, the number quickly increase to 19 ants, showing rapid pheromone reinforcement toward the shortest path.
- Across most of the 50 iterations, between 18 and 20 ants consistently used best path, indicating strong convergence and stable consensus.
- Key Takeaway: ACO can reliably converge to the optimal maze solution.

FUTURE DIRECTIONS

- Make ACO work for other graph sizes
- Test more with different iterations and size of ant colonies
- Alter pheromone factors

alpha: how strongly ants follow pheromone ~ 1.0

beta: controls how strongly ants prefer moves that appear closer to the goal ~ 2.0

evaporation: how strongly pheromone trails stay ~ 0.4

Q: controls how much pheromone successful ants drop ~ 50.0

exploreRate: controls how much ants will randomly explore ~ 0.05

maxSteps: prevents indefinite wandering ~ 4*(width+height)

e

ROAD BLOCKS

- We wanted to use multiple graph sizes to compare the algorithms, but the ACO algorithm uses pheromone factors which influence ant movement affecting output for medium size graphs.
- If we wanted ACO to work for small graphs, ants were not able to explore a larger size graph efficiently.
- If we did the opposite, ACO would be too deterministic for small graphs.
- Ultimately, even if our ACO worked for larger graphs, we would need a significant amount of data to show convergence.
- Since it takes an ant colony several iterations of the algorithm to build a stronger pheromone trail, we have to record several outcomes from each iteration, such as successful ant count, number of unique paths, and number of ants taking each unique path.

REFERENCES

- Introduction to Ant Colony Optimization - GeeksforGeeks
- Ant Colony Optimization - Intuition, Code & Visualization | Towards Data Science
- **Depth First Search or DFS for a Graph - GeeksforGeeks**
- **Breadth First Search or BFS for a Graph - GeeksforGeeks**
- **A* Search Algorithm - GeeksforGeeks**
- **The A* Algorithm: A Complete Guide | DataCamp**